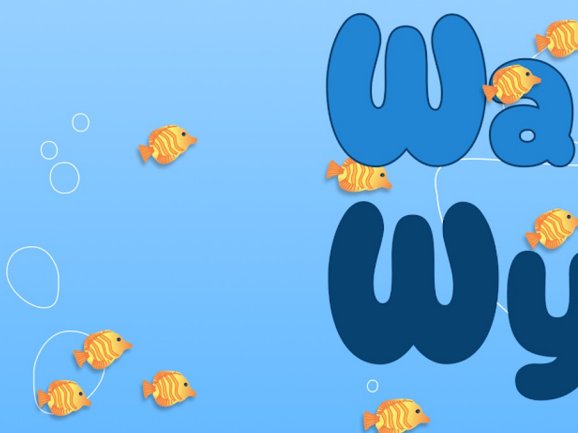
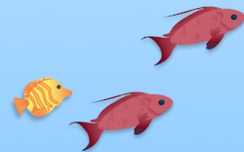


Wakacyjne Wyzwanie

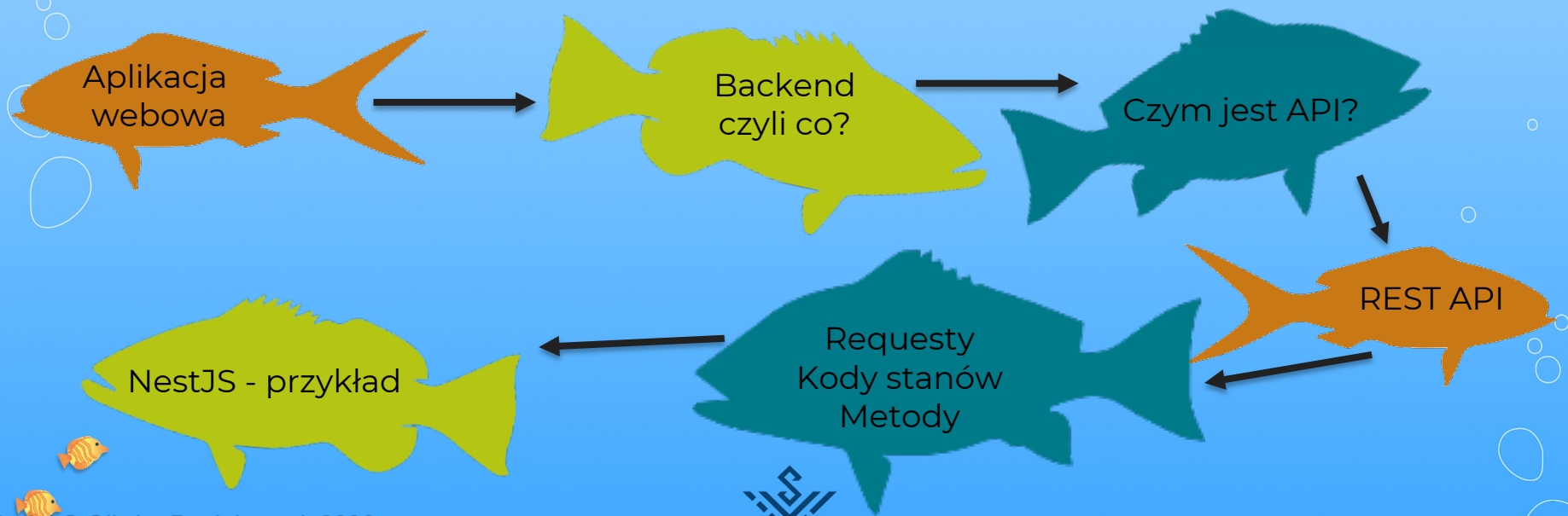


Z  KN Solvro

Ścieżka Backend #1

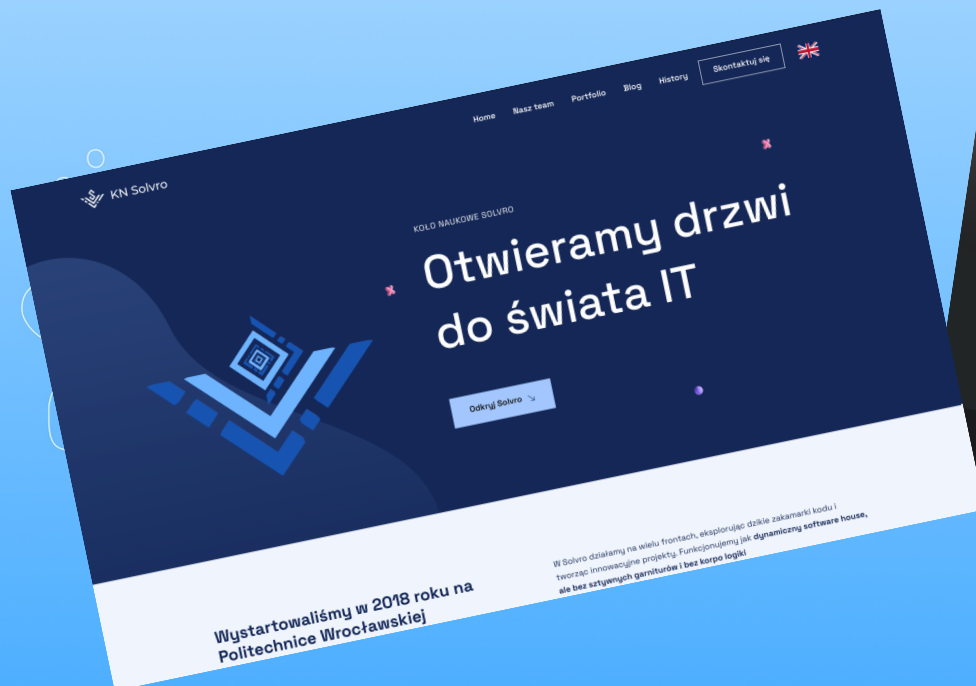


Podstawy API



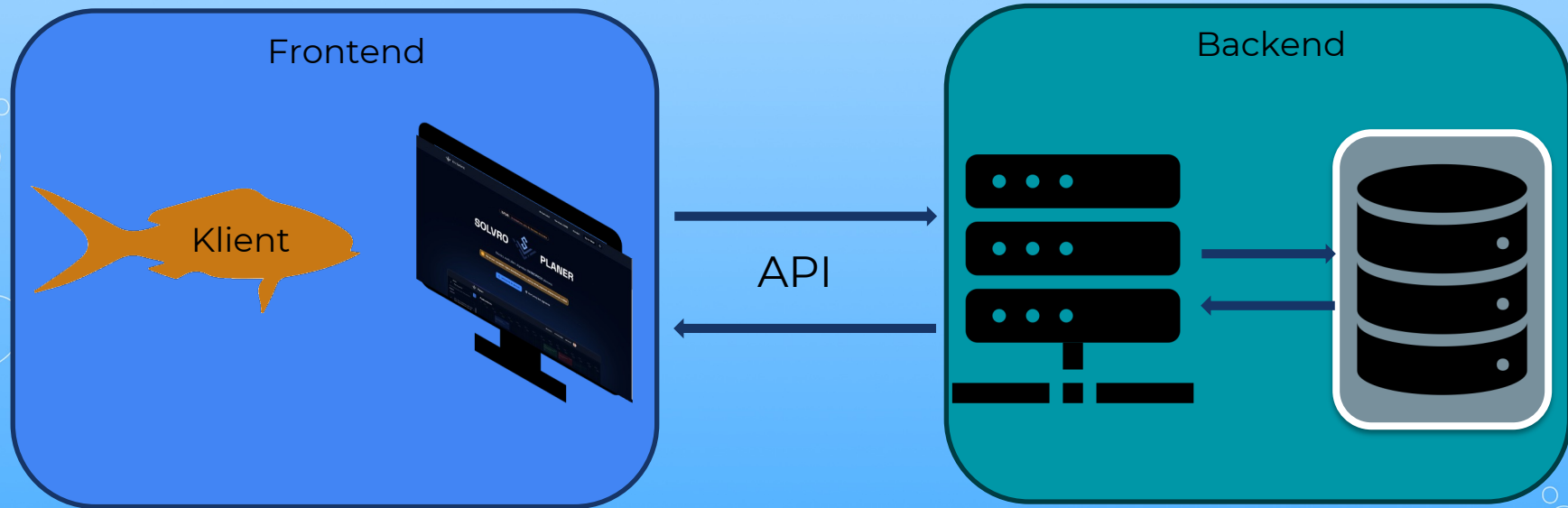
KN Solvro

Aplikacja webowa – czyli co?



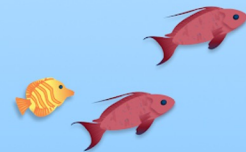
KN Solvro

Aplikacja webowa - budowa



KN Solvro

Czym się zajmujemy jako backend developerzy?



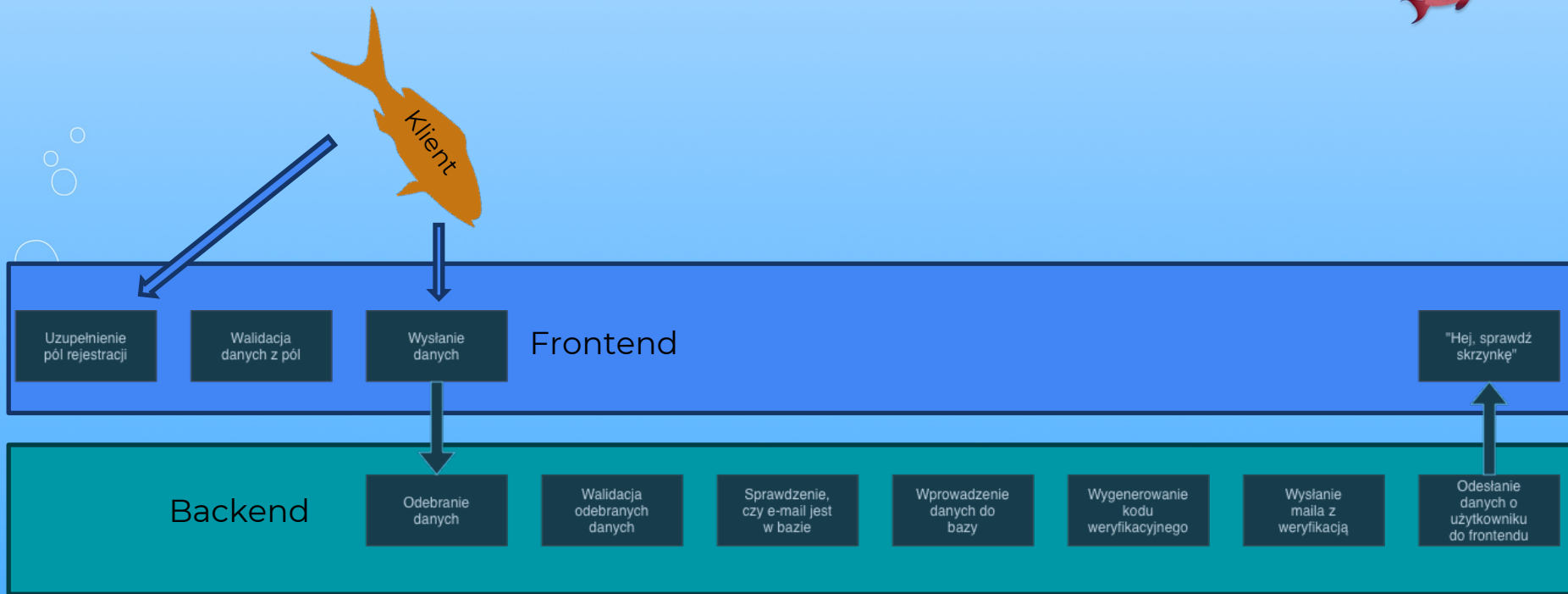
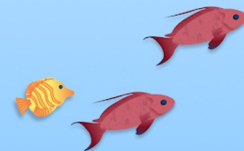
- Logika biznesowa aplikacji – reguły, według których aplikacja ma działać (np. przelewy bankowe, storki na ig)
- Dostęp do bazy danych – odczytywanie i zapisywanie do nich informacji, operowanie na nich, przekazywanie do frontendu
- Autoryzacja i uwierzytelnianie – sprawdzenie, jakie "prawa" ma dany użytkownik, dostosowanie jego możliwości działania do tych praw. Blokowanie niedozwolonych działań
- Walidacja (sprawdzanie) przesyłanych danych
- Maile/SMS'y weryfikacyjne
- Zadania powtarzalne/przypomnienia
- Wysyłanie powiadomień
- Łączenie i integracja z innymi serwisami i serwerami



KN Solvro

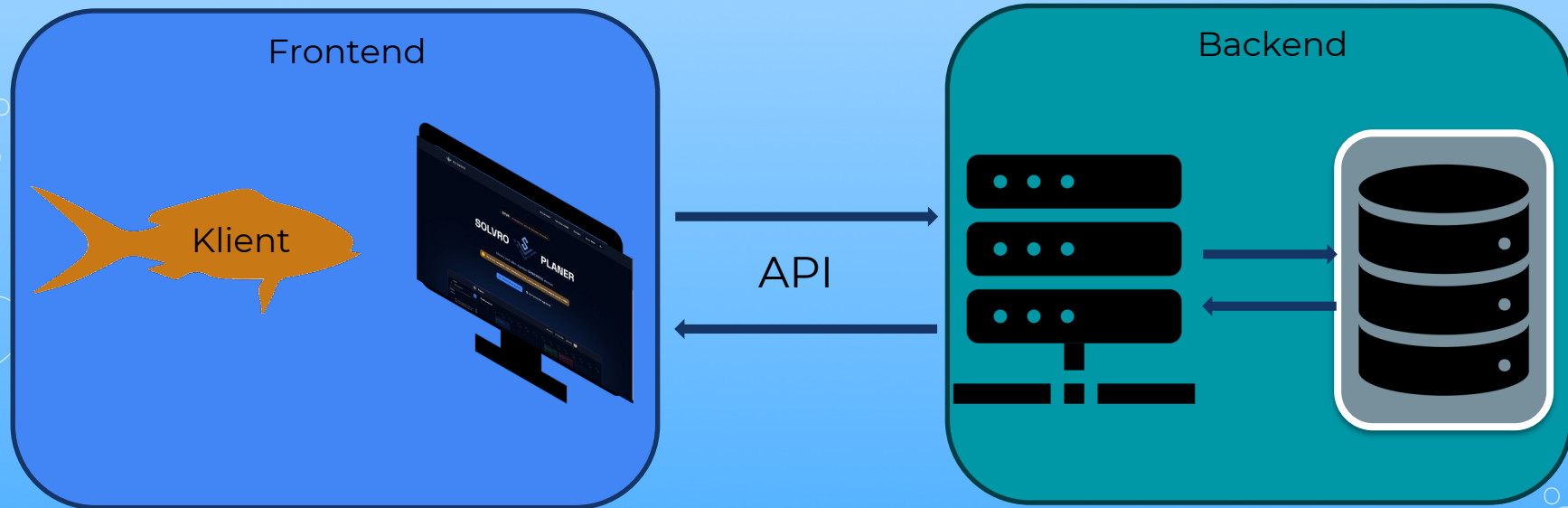
Kupa roboty w skrócie [*]

Backend w praktyce – rejestracja i logowanie



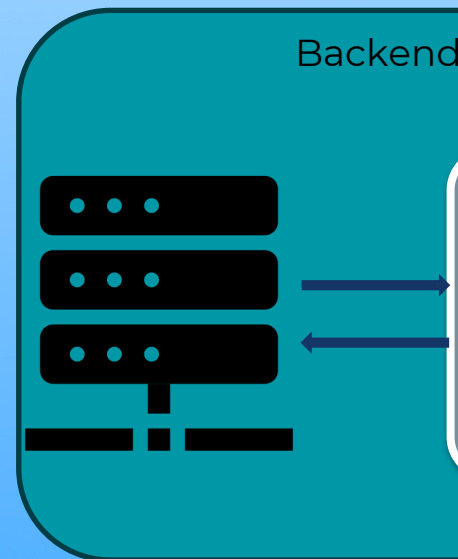
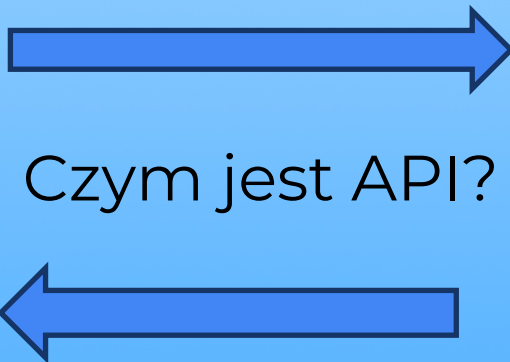
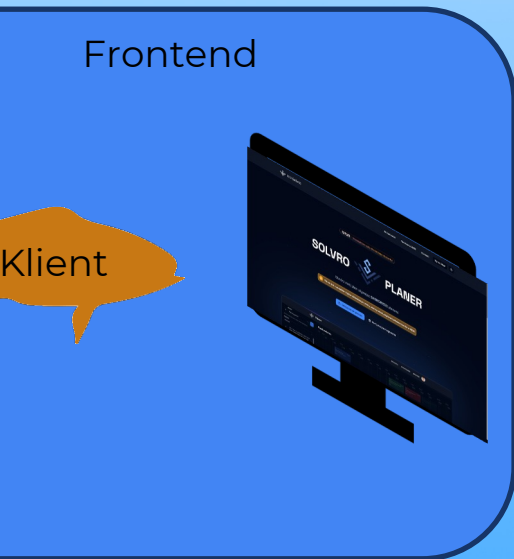
KN Solvro

Wróćmy tu

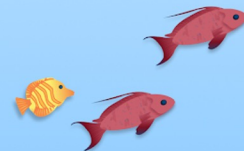


KN Solvro

Czym jest API?



API - Application Programming Interface



Czym jest API?

Stwórz nowy quiz

Uzupełnij podstawowe informacje i pytania, a następnie zapisz.

Tytuł

Podaj tytuł quizu

Opis

Podaj opis quizu

☐ Wielokrotny wybór (dla wszystkich pytań)

UI – interfejs dla człowieka

GET /api/user/ Get current user profile

Returns basic information about the currently authenticated user.

Parameters

No parameters

[Try it out](#)

Responses

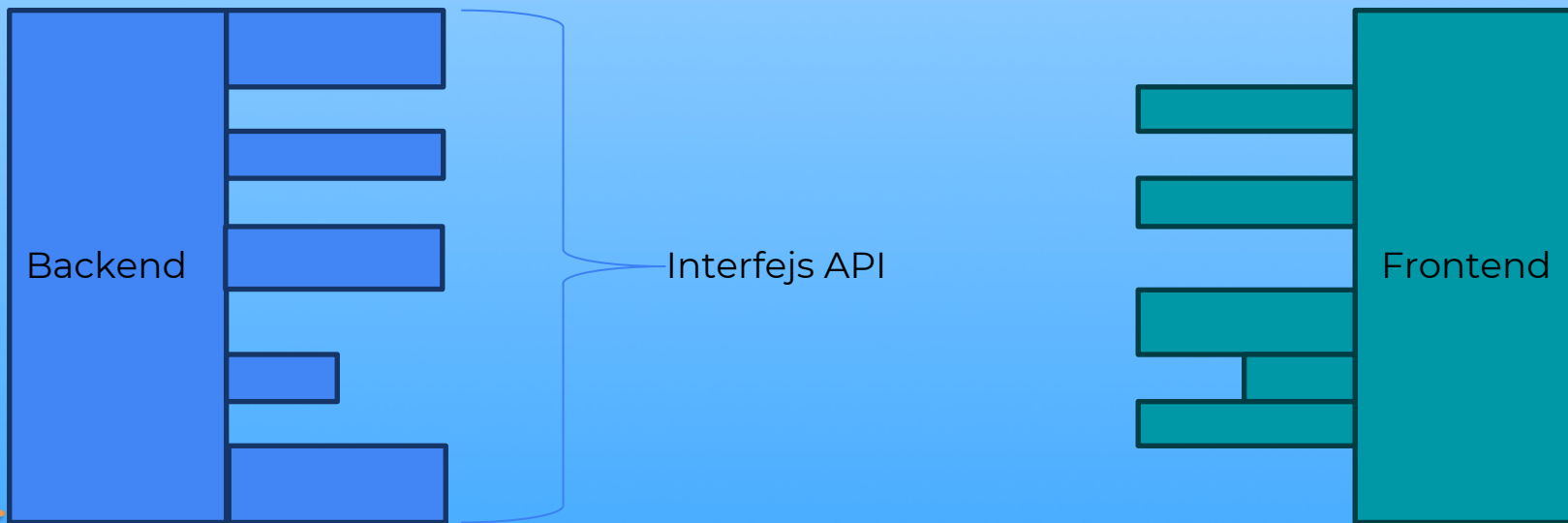
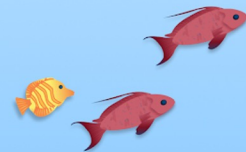
Code	Description	Links
200	Media type: application/json Controls Accept header: Example Value Schema <pre>{ "id": "3fa85f64-5717-4562-b3fc-2c963f66fef0", "email": "user@example.com", "full_name": "string", "is_superuser": true, "is_staff": true, "student_number": "string", "sex": "M", "gender": "string", "photo_url": "string", "override_photo_url": "string", "photo": "string", "hide_profile": true, "account_type": "guest", "account_level": "basic" }</pre>	No links

API – interfejs dla aplikacji



API - Application Programming Interface

Czym jest API?



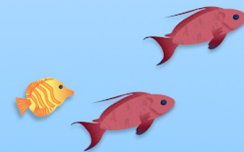
KN Solvro

API – brzmi ogólnie, jak to konkretnie ma wyglądać?

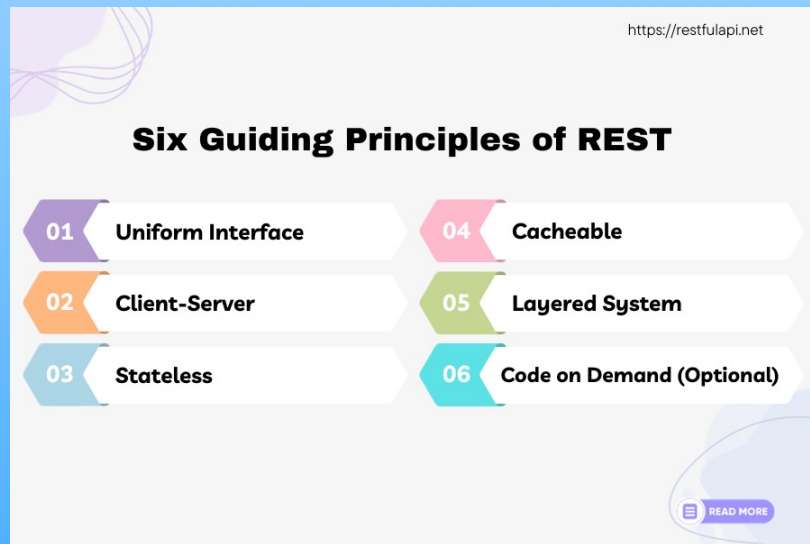
- My będziemy omawiać przede wszystkim REST API (zwanym też RESTful)
- To styl architektoniczny opisujący pisanie API opartego o protokół HTTP, czyli ten wykorzystywany m.in. przez przeglądarki internetowe do wyświetlania stron
- Inne rodzaje API to np. SOAP, GraphQL, WebSockets
- Jednak z perspektywy aplikacji internetowych są to rzadziej wykorzystywane architektury



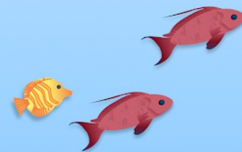
REST API - Representational State Transfer



- Można to opisać jako zasady, według których będziemy tworzyć nasze API
- Nie jest to sztywna specyfikacja, bardziej coś do czego powinniśmy dążyć
- Szczegółowa dokumentacja znajduje się tu: <https://restfulapi.net/>
- Teraz omówimy sobie część z tych zasad



REST API – Nazewnictwo ścieżek



- REST API opiera się na protokole HTTP, co za tym idzie jest to identyczny system ścieżek, co np. w aplikacjach internetowych

 <https://solvro.pwr.edu.pl/pl/portfolio/planer/>

- Ważne jest zatem odseparowanie ścieżek API od faktycznych podstron aplikacji – prefiks /api

Correct
`https://example.com/api/v3/products`
`https://example.com/api/v3/getAllProducts`
Wrong



KN Solvro

REST API – Wersjonowanie

- Kolejnym prefiksem po API powinien być prefiks wersji, zazwyczaj /v1, /v2, /v3 itd.

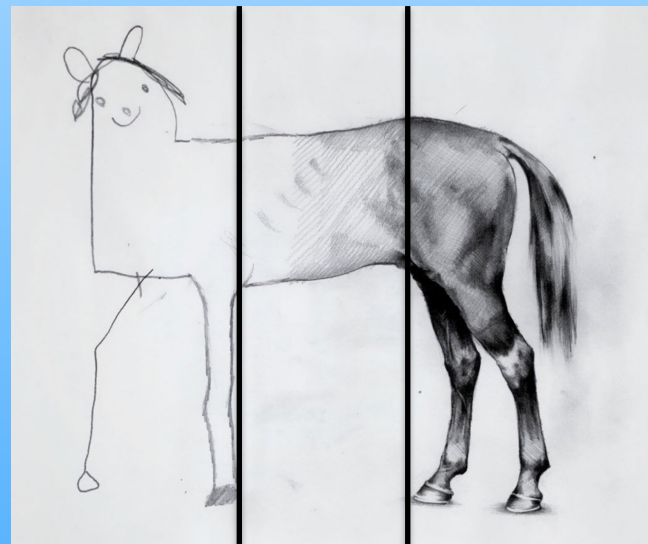
Correct

`https://example.com/api/v3/products`

`https://example.com/api/v3/getAllProducts`

Wrong

- Pozwala to płynnie publikować ważne zmiany w naszym systemie, dając czas innym na przystosowanie się do nich



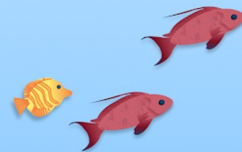
/api/v1

/api/v2

/api/v3



REST API – Headery



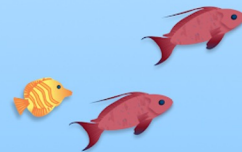
- Headery zawierają podstawowe informacje o naszym zapytaniu, pomagają serwerowi i klientowi zrozumieć jak przetwarzać żądanie

Header Name	Description
Authorization	Contains credentials (like tokens or basic auth) to authenticate a user.
Content-Type	Specifies the format of the request body (e.g., <code>application/json</code>).
Accept	Tells the server the format the client expects in the response.
User-Agent	Identifies the client (browser, app, etc.).
Cache-Control	Instructs caching behavior.
Cookie	Sends stored cookies to the server.



KN Solvro

REST API – Metody



- Metody możemy zrozumieć jako typy zapytań. Requests możemy podzielić ze względu na działania, które wykonują, i w zależności od tych działań dobrać odpowiednią metodę zapytania

HTTP Request Methods

	GET	Retrieves data or resource from a specified URL. It is used to retrieve information without modifying it.
	POST	Submit data or creates a new resource on the server. It is used to send data to be processed by the server. It often results in the creation of a new resource on the server.
	PUT	Updates the existing resource with the new data. It replaces the entire resource or creates it if it does not exist.
	DELETE	Deletes the specified resource from the server.
	PATCH	Partially updates the existing resource with the provided data. PATCH request does not create a new resource if the specified resource does not exist on the server.
	HEAD	Retrieve only the headers of a response. It is used to check the status or headers of a resource without fetching the actual content.
	OPTIONS	Fetch or Retrieve the allowed methods and other information of the specified resource.
	TRACE	The TRACE method echoes back the received request to the client, allowing clients to inspect the request and see any modifications or additions made by intermediaries. It is mainly used for the diagnostic purposes.
	CONNECT	Converts the request connection to a transparent TCP/IP tunnel, commonly used for establishing secure SSL/TLS connections through proxies.

TestKarts.com

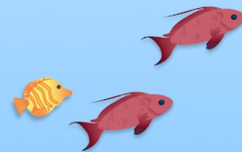
actual methods

mental disorders



KN Solvro

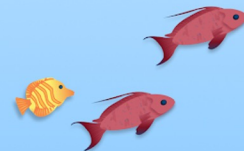
REST API – Metody



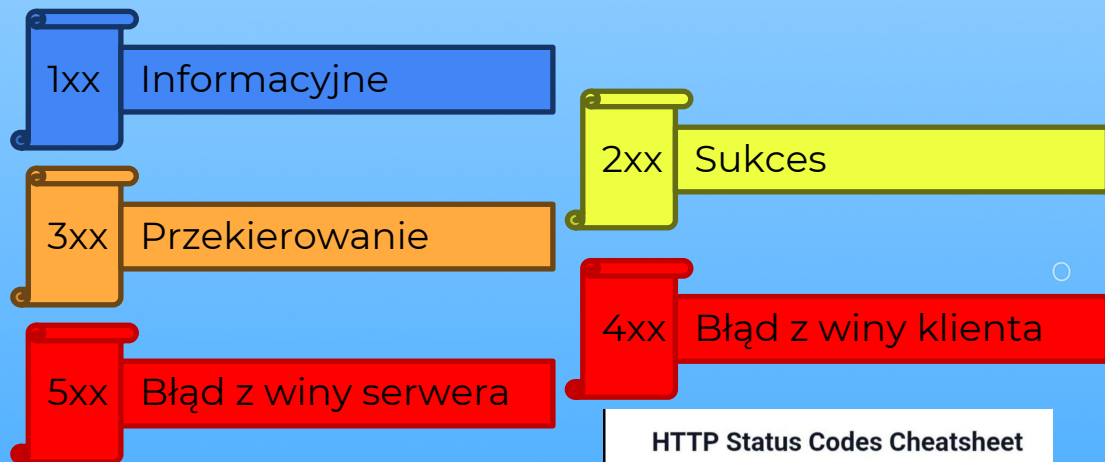
- Metody możemy zrozumieć jako typy zapytań. Requests możemy podzielić ze względu na działania, które wykonują, i w zależności od tych działań dobrać odpowiednią metodę zapytania



REST API – Status codes



- Kody informują nas o tym, w jaki sposób serwer odpowiedział na nasze zapytanie. Czy się powiodło, a jak nie to co zawiodło?



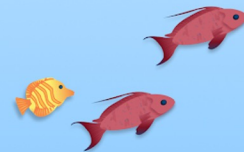
HTTP Status Codes Cheatsheet

1xx : Hold on
2xx : Here you go
3xx : Go away
4xx : You f*cked up
5xx : We f*cked up



KN Solvro

REST API – Kody 2xx



200
OK

Jest okej

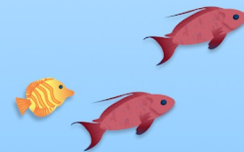


201
Created

Jest okej, ale to POST



REST API – Kody 4xx



400
Bad Request



401
Unauthorized



403
Forbidden



404
Not Found

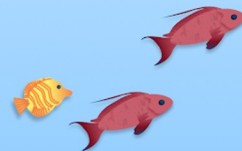


429
Too Many Requests

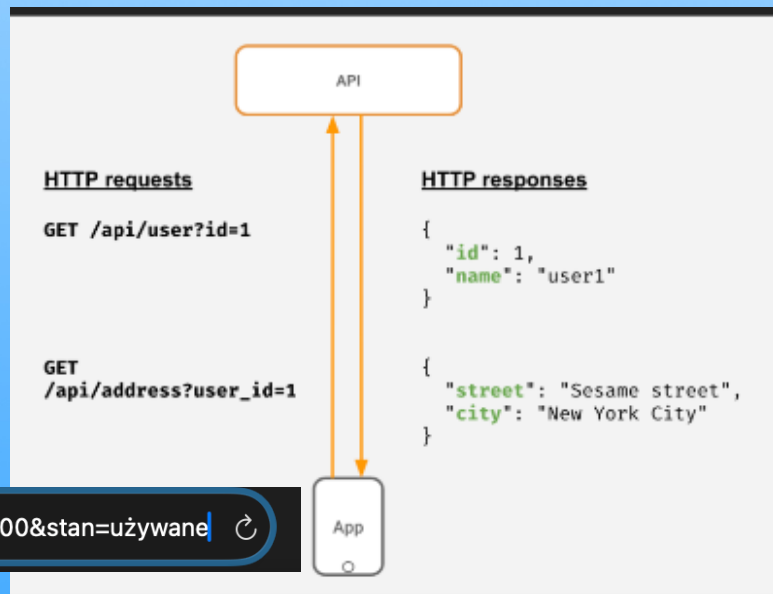


418
I'm a teapot

REST API – Parametry (Query Params)



- Dane możemy przekazywać razem z adresem naszego zapytania
- Zazwyczaj używamy tego z metodą GET
- Przekazujemy dane o np.
 - Filtrowaniu wyników
 - Sortowaniu
 - Paginacji
- Czasem jeszcze są to dane po prostu z formularzy

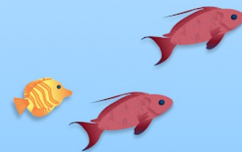


[a s://allegro.pl/kategoria/podzespolo-komputerowe-karty-graficzne-260019?price_to=3000&stan=uzywane](https://allegro.pl/kategoria/podzespolo-komputerowe-karty-graficzne-260019?price_to=3000&stan=uzywane)



KN Solvro

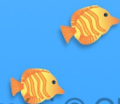
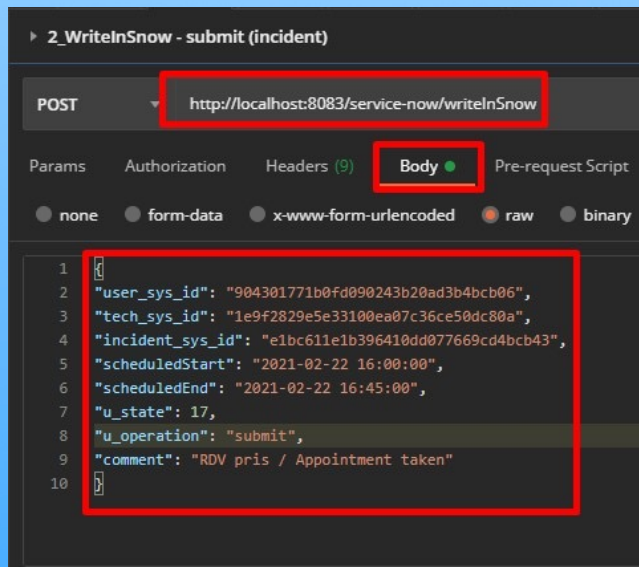
REST API – Body



- Główna forma przesyłania danych dotyczących naszego żądania
- Zazwyczaj używamy formatu JSON

```
{
  "title": "New Dropdown (multi select) column",
  "type": "TEXT_NUMBER",
  "options": ["First", "Second", "Third"],
  "index": 6
}
```

Body



KN Solvro

REST API – Podsumowanie

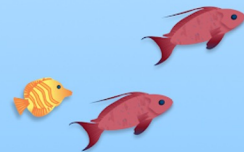
Therapist: Do you get enough rest?

Me: Of course. REST is a lifestyle.

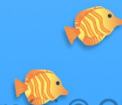


KN Solvro

NestJS

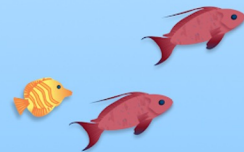


- Bardziej praktyczna część kursu będzie przedstawiana na frameworku NestJS
- Jego też będziecie używać do rozwiązywania zadań



KN Solvro

NestJS – instalacja i pierwszy projekt

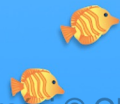


- Aby móc utworzyć pierwszy projekt, musimy zainstalować paczkę z CLI

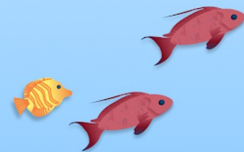
```
$ npm i -g @nestjs/cli
```

- Następnie możemy utworzyć nasz pierwszy projekt za pomocą **nest new**

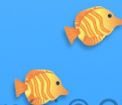
```
$ nest new project-name
```



NestJS – struktura plików

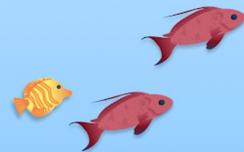


- Przejdźmy do vs code'a, to sobie omówimy po kolei wszystko
- Prezentacja którą dostaniecie zostanie uzupełniona o te informacje



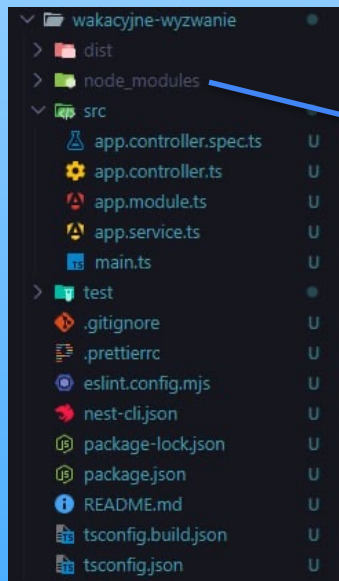
KN Solvro

NestJS – struktura projektu

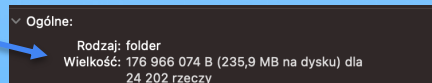


- Po utworzeniu projektu utworzy nam się folder z jego nazwą i taką zawartością:

- Dist pomijamy – to folder na build'y projektu
- node_modules to zewnętrzne zależności
- W src/ kodzimy
- W test/ testujemy



- .gitignore – plik z ignorowanymi przez gh ścieżkami

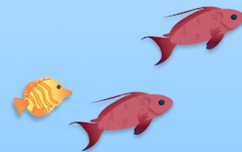


- .prettier, eslint – ładny kodzik i porządky
- nest-cli – jakaś konfiguracja CLI
- package.json, package-lock.json – pliki projektu i jego zależności
- README.md – automatycznie generowane readme
- tsconfig – konfiguracja typescript'a



KN Solvro

NestJS – controller



- W kontrolerach ustalamy, jak aplikacja ma odpowiadać na nasze zapytania

W konstruktor
wrzucamy serwisy, z
których ten
kontroler korzysta

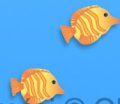
```
app.controller.ts

import { Controller, Get } from '@nestjs/common';
import { AppService } from '../app.service';

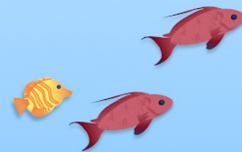
@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

Tutaj definiujemy
metodę, która
będzie uruchamiana
gdy serwer otrzyma
zapytanie metodą
GET



NestJS – service



- W serwisach definiujemy już faktyczną logikę i funkcjonalność serwera za pomocą metod, które potem uruchamiamy z poziomu kontrolera

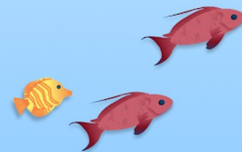
```
app.service.ts

import { Injectable } from '@nestjs/common';

@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}
```



NestJS - module



- Moduły to połączenie kontrolera i działających z nim serwisów w jedno

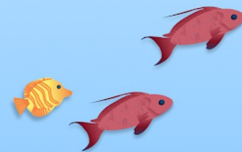
```
app.module.ts

import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';

@Module({
  imports: [],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```



NestJS - main.ts



- To główny plik naszego projektu, uruchamia on serwer i wskazuje główny moduł

```
main.ts

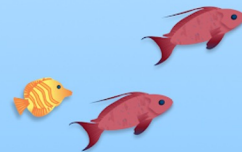
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}

bootstrap();
```



NestJS – machina ruszyła



- Aby uruchomić nasz projekt, używamy polecenia w konsoli:

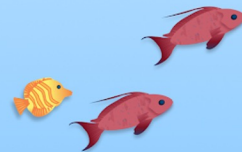
```
$ npm run start
```

- Dopisując :dev uruchamiamy serwer w trybie deweloperskim, co pozwala monitorować zmiany w plikach, i na bieżąco aktualizować zmiany na podstawie tego co kodzimy

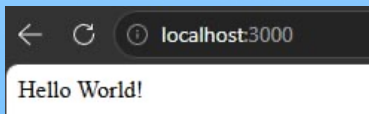
```
$ npm run start:dev
```



NestJS – machina ruszyła



- Wchodząc na adres `http://localhost:3000` (lub inny port który został sprecyzowany w `.env`) możemy zobaczyć efekty naszych starań

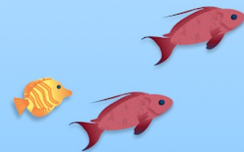


- Wygląda trochę jak z 95, ale może, bo to backend, niech się frontendowcy z CSS'em męczą



KN Solvro

Backendowe narzędzia deweloperskie



- Testowanie API w przeglądarce bywa męczące, lepiej użyć specjalnie do tego przygotowanych narzędzi



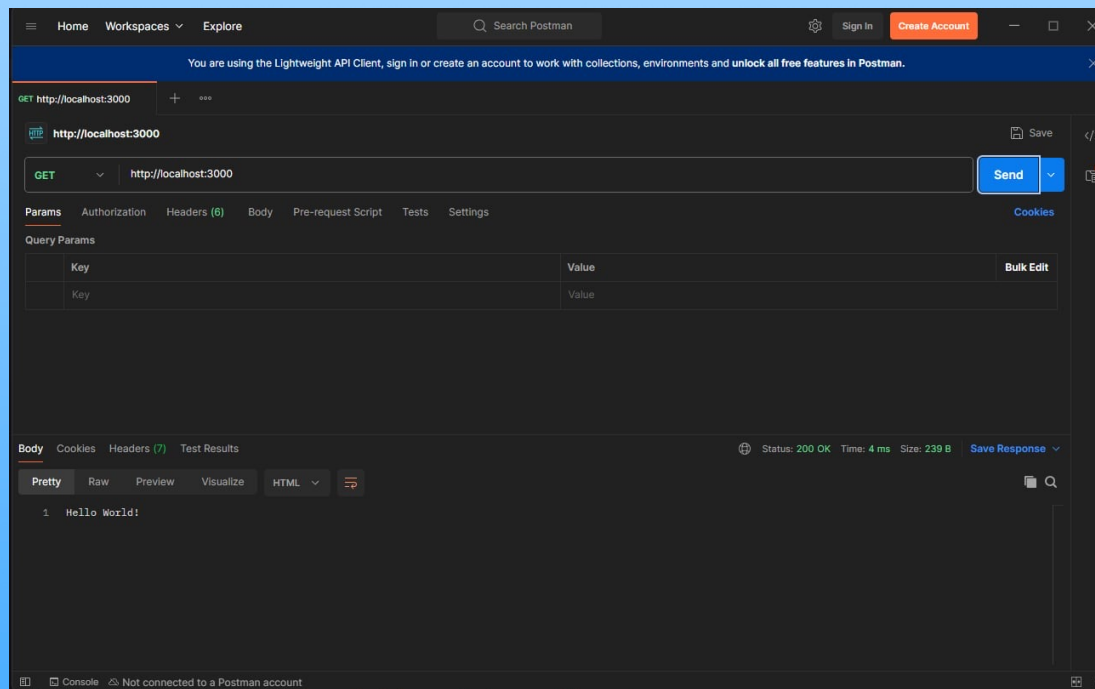
Postman



Insomnia

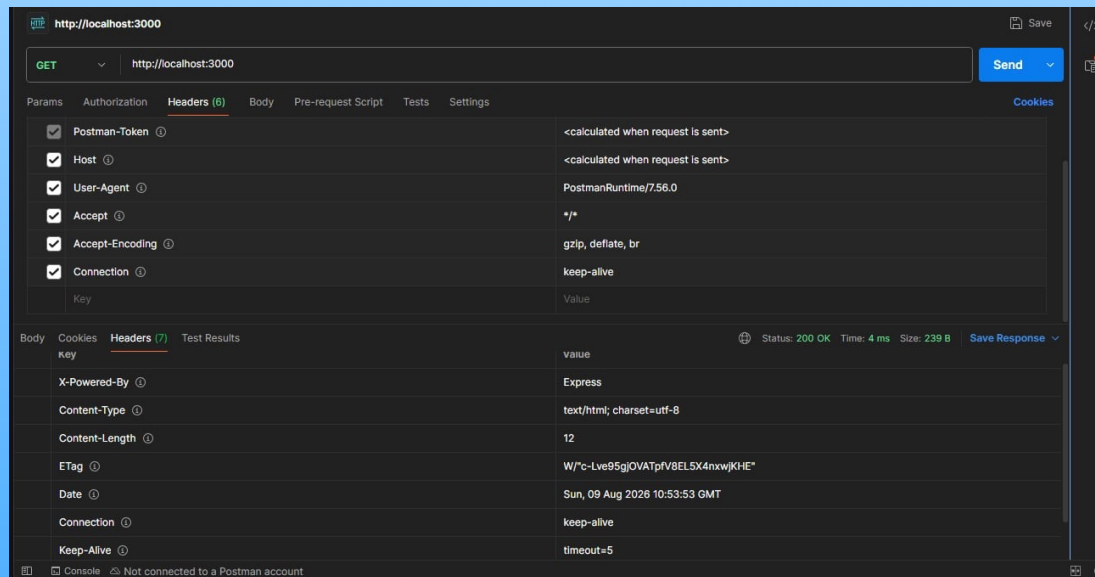
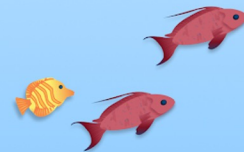


Postman – sprawdzamy czy nasz backend działa



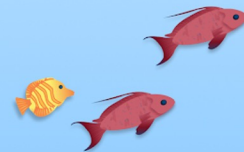
KN Solvro

Postman – sprawdzamy czy nasz backend działa



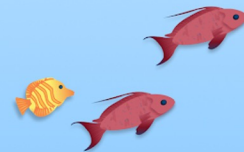
Zadanie domowe i materiały

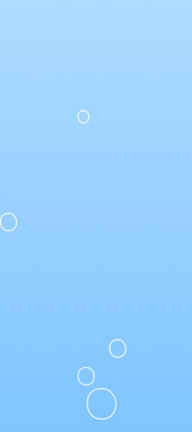
#będzie



KN Solvro

Pytania? Wolne wnioski?





Dzięki za uwagę

